SDLC ASSIGNMENT: SMART ATTEND MOBILE APP

SDLC & DevOps Fundamentals

University
Lovely Professional University
Phagwara, Punjab

Mouli Banerjee
Reg No.: 12300455

Project Scenario (Analysis)

The university startup is developing a mobile application called **Smart-Attend**, which is designed to automate the process of student attendance. The application uses **Geo-fencing technology** to detect when a student enters the classroom area. Geo-fencing works by creating a virtual boundary around a specific location. When a student's mobile device enters this predefined classroom boundary, the system automatically detects the location and marks the student's attendance.

This system aims to reduce the manual effort required from teachers and improve the accuracy of attendance records. It also helps in maintaining real-time attendance data for students and teachers.

However, during the middle of the development process, the **Dean introduces a new requirement**. The Dean states that some students may attempt **proxy attendance**, where one student carries another student's device to mark attendance even if the student is not physically present in the classroom.

To solve this issue, the application must now include **Biometric Face ID verification**. This means the system must verify the identity of the student using facial recognition before marking attendance.

This new requirement significantly affects the system architecture because biometric authentication requires additional components such as:

- ✓ Camera access from the mobile device
- ✓ Facial recognition processing
- ✓ Identity verification algorithms
- ✓ Additional security layers

Since this requirement is introduced **after the development phase has already started**, it creates serious challenges for the development process. This situation highlights the limitations of traditional development models like the **Waterfall Model**, which are not designed to easily accommodate changes during development.

Waterfall Failure Analysis

The **Waterfall Model** is a traditional software development approach that follows a linear and sequential process. In this model, each phase of development must be completed before the next phase begins.

The major phases of the Waterfall Model include:

1. Requirements Analysis
2. System Design
3. Development
4. Testing
5. Deployment

This model works effectively when the project requirements are clearly defined at the beginning and do not change during the development process. However, in the Smart Attend project, the introduction of **Biometric Face ID verification halfway through development creates significant challenges for the Waterfall approach**.

The following reasons explain why the Waterfall model would cause a crisis in this situation:

1. **Rigidity of the Waterfall Model**

One of the biggest limitations of the Waterfall model is its **rigid structure**. Once the project moves from the requirements phase to the design and development phases, it becomes extremely difficult to go back and modify earlier stages.

In the Smart-Attend project, the system was initially designed only for **Geo-fencing based attendance tracking**. The architecture, authentication system, and database structure were all built around location-based detection.

When the Dean introduces the **Face ID biometric requirement**, the development team would need to revisit the requirement and design phases to integrate biometric authentication features.

This would require several major modifications, including:

- ✓ Updating the authentication mechanism
- ✓ Integrating camera functionality into the mobile application

- ✓ Implementing facial recognition algorithms
- ✓ Modifying the database structure to support biometric verification

Because the Waterfall model does not easily support changes after development begins, the team would face serious difficulties implementing the new requirement.

2. **<u>High Cost of Change</u>**

Another major disadvantage of the Waterfall model is the **high cost of making changes during the development phase**.

In Waterfall development, the system design is treated as a **fixed blueprint**. Once development begins, modifying the design requires significant effort and resources.

Adding **Biometric Face ID verification** at this stage would require the development team to perform multiple major changes, such as:

- ❖ Redesigning the user authentication system
- ❖ Integrating facial recognition algorithms
- ❖ Adding camera permission handling
- ❖ Updating backend services and APIs
- ❖ Modifying the database structure
- ❖ Implementing additional security validations

These changes would result in:

- ✓ Increased development cost
- ✓ Extended project timeline
- ✓ Additional testing requirements
- ✓ Reallocation of development resources

Therefore, introducing such a major change in the Waterfall model becomes **expensive and time-consuming**.

3. **<u>Delayed Testing and Integration Issues</u>**

In the Waterfall model, the **testing phase occurs only after the development phase is completed**. This means that integration problems are often discovered very late in the development process.

If biometric authentication is added late in development, the team may encounter several issues during testing, such as:

- Facial recognition accuracy problems
- Camera permission conflicts on different mobile devices
- Compatibility issues across different operating systems
- Synchronization problems between Geo-fencing and biometric verification
- Performance issues due to facial recognition processing

If these issues are discovered **after months of development**, fixing them could significantly delay the release of the application.

This delayed feedback cycle is one of the major weaknesses of the Waterfall model.

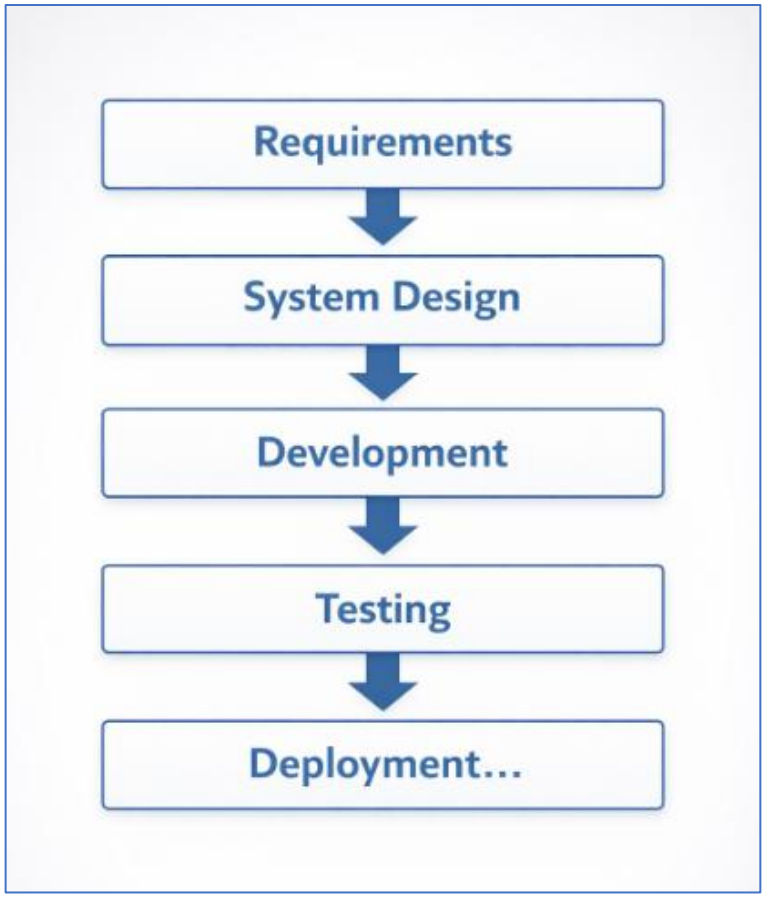

Figure 1: Waterfall Development Model

Agile Pivot Using Scrum Framework

To address the challenges caused by changing requirements, the development team decides to adopt the **Agile Scrum framework**.

Agile development focuses on **flexibility, continuous improvement, and iterative development**. Instead of developing the entire system in one long development cycle, the project is divided into smaller cycles called **Sprints**.

Each sprint typically lasts **two weeks** and produces a working version of the product. The Scrum framework involves several stages, including:

- Product Backlog creation
- Sprint Planning
- Sprint Development
- Sprint Review
- Sprint Retrospective

This approach allows the development team to continuously receive feedback and adapt the system based on new requirements.

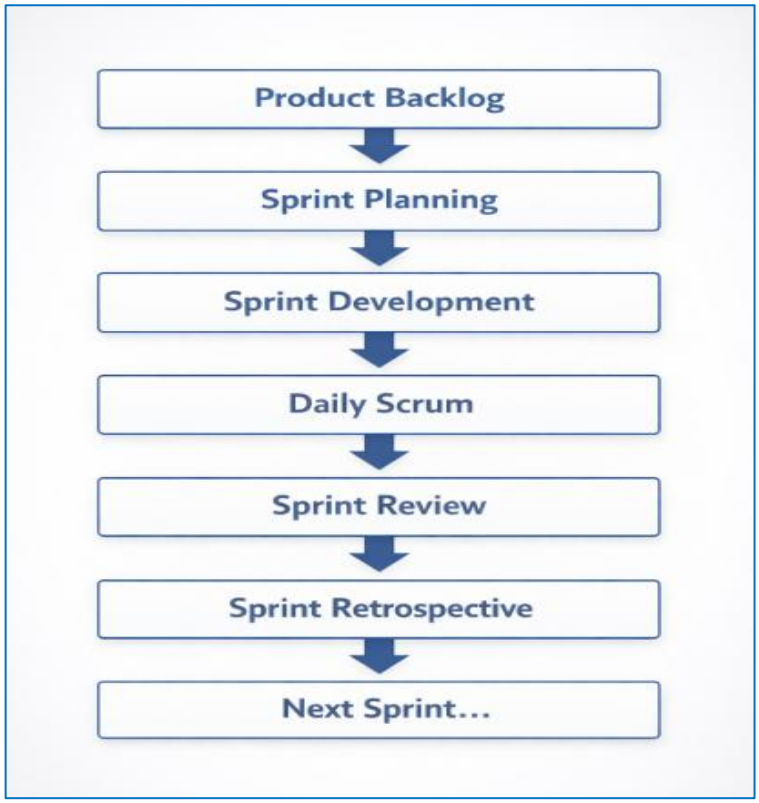

Figure 2: Agile Scrum Development Cycle

Sprint Planning

To successfully develop the Smart-Attend application, the project is divided into **two development sprints**.

Each sprint focuses on delivering a set of specific features within a short period of time.

- **Sprint 1 – Minimum Viable Product (MVP)**

  The goal of Sprint 1 is to build the **Minimum Viable Product (MVP)** of the Smart-Attend application. The MVP focuses on implementing the core functionality of the attendance system.

  The main features developed during Sprint 1 include:
  - ✓ Development of the basic mobile application interface
  - ✓ Implementation of the student login system using university ID
  - ✓ Development of Geo-fencing logic to detect classroom entry
  - ✓ Real-time attendance monitoring for teachers
  - ✓ Notification system to inform students when their attendance is marked

  At the end of this sprint, the team delivers a **working prototype of the Smart Attend system**.

- **Sprint 2 – Feature Update (Biometric Integration)**

  The objective of Sprint 2 is to improve the security and reliability of the system by integrating **Biometric Face ID verification**.

  The key features developed during Sprint 2 include:
  - ✓ Integration of Face ID biometric authentication
  - ✓ Camera permission management within the mobile application
  - ✓ Facial recognition verification workflow
  - ✓ Integration of biometric verification with Geo-fencing attendance detection
  - ✓ Development of an improved reporting dashboard for teachers and administrators
  - ✓ Bug fixing and performance optimization

  This sprint ensures that the Smart-Attend system can effectively **prevent proxy attendance**.

Sprint Backlog

The **Sprint Backlog** is a list of tasks that the development team commits to completing during the sprint.

The Sprint Backlog for Sprint 1 includes the following user stories.

Task ID	User Story	Priority	Estimated Hours
US01	As a student, I want to login via university ID so that I can access my profile.	High	4
US02	As a teacher, I want to view a real-time list of present students so that I can monitor attendance easily.	High	6
US03	As a system, I want to detect if a student is within 10 meters of the classroom so that attendance can be automatically recorded.	High	8
US04	As a student, I want to receive a notification when my attendance is marked so that I know it has been successfully recorded.	Medium	3
US05	As an admin, I want to manually override attendance records so that technical errors can be corrected.	Low	4

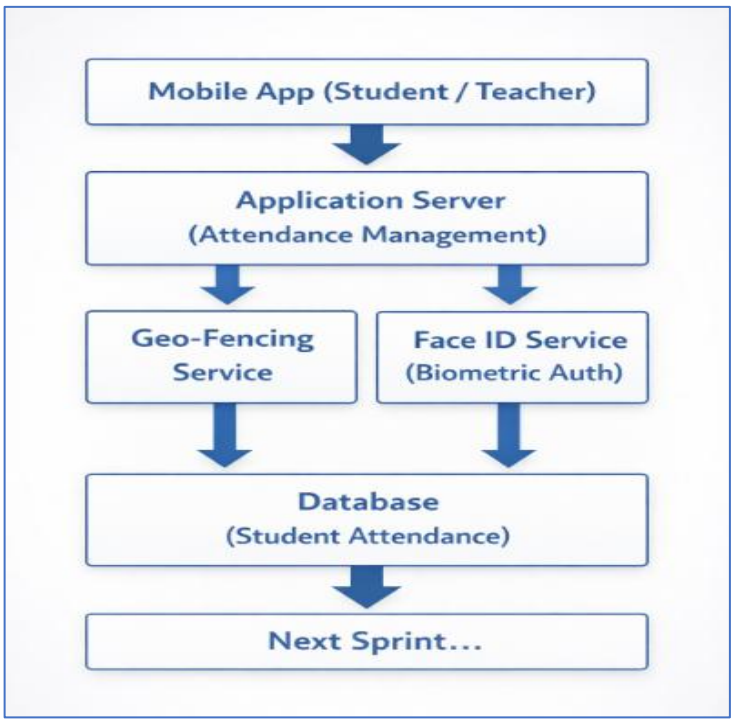

Figure 3: Smart-Attend System Architecture

CI/CD Explanation

CI/CD stands for Continuous Integration and Continuous Deployment. It is a development practice that automates the process of building, testing, and deploying software updates.

In the Smart-Attend project, CI/CD would provide several important benefits, including:

- Faster delivery of new features and updates
- Automatic testing of biometric authentication modules
- Reduced risk of deployment errors
- Continuous delivery of improvements to students and teachers
- Faster detection and resolution of bugs

By using CI/CD, the development team can ensure that every sprint update is automatically tested and deployed to the application. This allows the Smart-Attend system to continuously improve while maintaining reliability and performance.

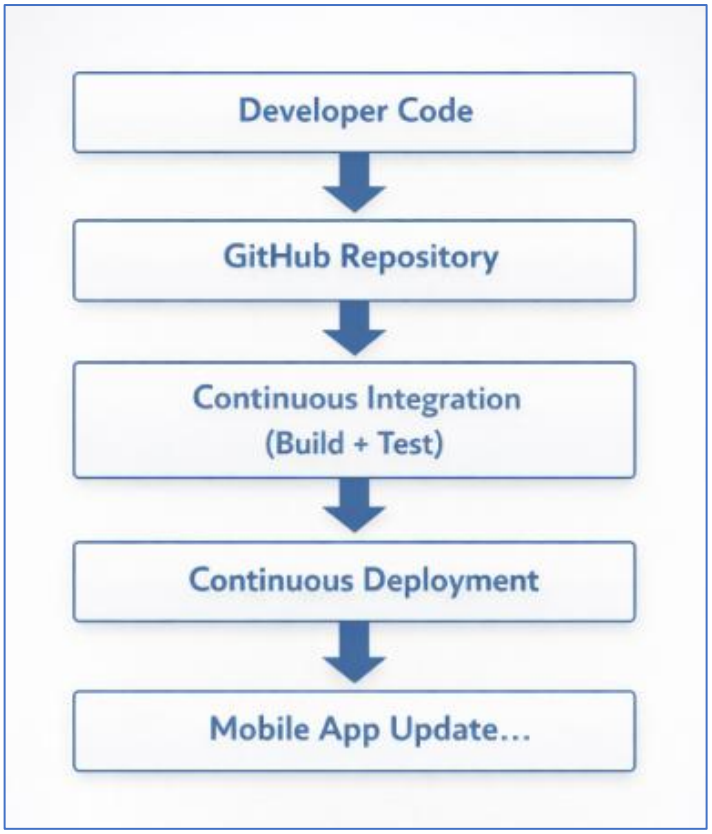

Figure 4: CI/CD Deployment Pipeline